Warehouse NeoLab — résumé technique pour l'hébergeur

Document court à destination d'une équipe d'infrastructure (ex. Valeria) qui doit évaluer une demande d'hébergement, sans connaître le projet. Détails complets dans `docs/WAREHOUSE_RELEASE_V1.md` et les autres guides du dossier `docs/`.

En une phrase

Une base PostgreSQL 16 + PostGIS qui centralise des données océanographiques (EcoTaxa, EcoPart, profils CTD, données de filets) pour le laboratoire NeoLab, avec des rôles d'accès en lecture déjà conçus et testés.

Besoins techniques minimaux

Élément	Valeur
Moteur	PostgreSQL **16**
Extension	**PostGIS** 3.4+ (calculs géographiques ; sans elle, tout fonctionne sauf le filtre spatial)
Stockage	20–50 Go au départ ; viser jusqu'à ~15 millions de lignes à terme
Droits du compte propriétaire	`LOGIN`, `CREATEROLE` (pour créer des comptes de lecture), idéalement activation de PostGIS par un admin si le propriétaire n'est pas superuser
Réseau	Accès depuis le réseau UL / VPN. **Pas besoin d'exposition publique.** Connexion chiffrée (TLS) souhaitable si accès distant
Sauvegarde	`pg_dump` standard, format custom, recommandé quotidien
Système	Linux, Docker disponible (optionnel mais simplifie le déploiement)

Déploiement

Deux façons possibles, au choix de l'hébergeur :

1. Docker (le plus simple) — une image `postgis/postgis:16-3.4` + scripts d'initialisation fournis :

```
docker compose -f docker-compose.warehouse.yml up -d
```

Crée le schéma, les rôles, le contrôle d'accès, et vérifie automatiquement que tout est prêt.

2. PostgreSQL existant — un script Python unique applique tout (migrations, rôles, vérifications) sur une base déjà créée par l'hébergeur :

```
python scripts/deploy_warehouse_postgres.py --check
```

Rejouable sans risque : une base existante n'est jamais réécrite, seules les migrations additives s'appliquent.

Dans les deux cas, un script de vérification (`--verify`) confirme en une commande que tout est conforme (rôles créés, sécurité active, pas de contournement) avant d'ouvrir l'accès à l'équipe.

Modèle d'accès (déjà conçu, pas à définir par l'hébergeur)

- **Un seul compte propriétaire** (`neolab`), qui charge les données et gère les rôles.
- **Comptes de lecture par personne**, créés à la demande, avec un périmètre précis (source de données, contrat, zone géographique, années). Un compte sans droit explicite ne voit rien.
- Les comptes de lecture sont `NOSUPERUSER`, ne peuvent pas écrire, ont un délai de requête limité à 30 s.
- Les mots de passe transitent uniquement par variables d'environnement, jamais dans le code ni en ligne de commande.

L'hébergeur n'a donc **pas besoin de concevoir de politique d'accès** : il fournit un compte propriétaire avec `CREATEROLE`, le reste est géré par le laboratoire via des scripts déjà écrits et testés.

Ce qui a été vérifié avant cette demande

- Déploiement Docker complet exécuté et validé (init, rôles, sécurité, PostGIS, interface de consultation).
- Déploiement sur un PostgreSQL "générique" (sans Docker) également testé, y compris avec un propriétaire non-superuser (cas typique d'un serveur mutualisé/géré).
- 123 tests automatisés, dont plusieurs contre une vraie base PostgreSQL.
- Chaîne de chargement des données (EcoTaxa → EcoPart → CTD → filets) exécutée de bout en bout sur un jeu de test.

Ce qui reste à faire une fois le serveur fourni

- Charger les données réelles (nécessite des identifiants EcoTaxa, pas encore disponibles dans cette phase).
- Test avec un certificat TLS d'une vraie autorité (seul `localhost` a été testé jusqu'ici).
- Volume réel (~15 millions de lignes), extrapolé mais non mesuré en grandeur réelle.

Questions probables de l'hébergeur, avec réponses

Question	Réponse
Combien de connexions simultanées ?	Faible (quelques chercheurs), pas de charge haute fréquence prévue
Faut-il exposer un port sur Internet ?	Non — accès réseau interne / VPN suffit
Qui gère les sauvegardes ?	À discuter — le labo peut gérer `pg_dump` lui-même si l'hébergeur fournit juste le stockage
Le schéma change-t-il souvent ?	Rarement ; les migrations sont scriptées et rejouables sans perte de données
Support requis de l'hébergeur ?	Minimal : provisionner le serveur/la base, donner un compte avec `CREATEROLE`, activer PostGIS si possible